

```

    int i;
    char ch[2];
};
union a key;

key.i = 512;
printf ( "\nkey.i = %d", key.i );
printf ( "\nkey.ch[0] = %d", key.ch[0] );
printf ( "\nkey.ch[1] = %d", key.ch[1] );
}

```

And here is the output...

```

key.i = 512
key.ch[0] = 0
key.ch[1] = 2

```

As you can see, first we declared a data type of the type **union a**, and then a variable **key** to be of the type **union a**. This is similar to the way we first declare the structure type and then the structure variables. Also, the union elements are accessed exactly the same way in which the structure elements are accessed, using a '.' operator. However, the similarity ends here. To illustrate this let us compare the following data types:

```

struct a
{
    int i;
    char ch[2];
};
struct a key;

```

This data type would occupy 4 bytes in memory, 2 for **key.i** and one each for **key.ch[0]** and **key.ch[1]**, as shown in Figure 15.1.

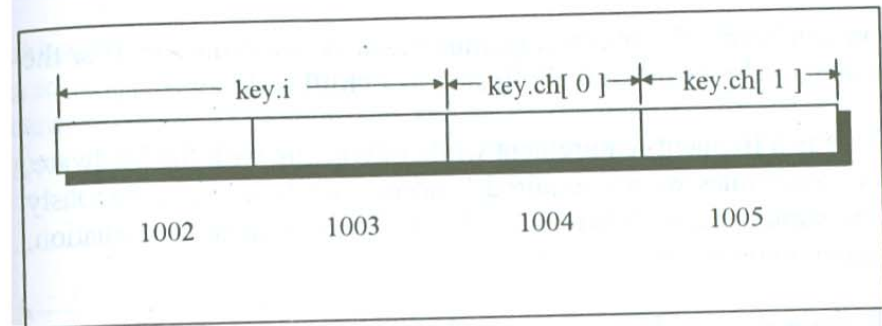


Figure 15.1

Now we declare a similar data type, but instead of using a structure we use a union.

```

union a
{
    int i;
    char ch[2];
};
union a key;

```

Representation of this data type in memory is shown in Figure 15.2.

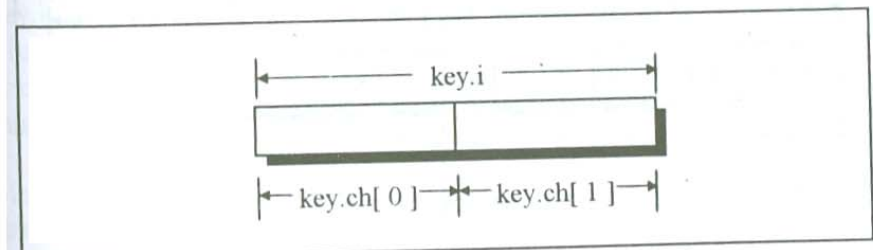


Figure 15.2

As shown in Figure 15.2, the union occupies only 2 bytes in memory. Note that the same memory locations which are used for **key.ch[0]** and **key.ch[1]**. It means that the memory locations used by **key.i** can also be accessed using **key.ch[0]** and **key.ch[1]**. What purpose does this serve? Well, now